

ResearchIQ - Complete Project Documentation (Version 1.0)

Project Overview

ResearchIQ is an AI-powered academic research assistant that automates the literature review process. It helps students and researchers quickly find relevant papers, understand them through AI summaries, identify research gaps, and generate actionable project ideas.

Live Application: <https://research-iq-flax.vercel.app>
Backend API: <https://researchiq-backend-hbzb.onrender.com>
GitHub Repository: <https://github.com/Karishma1105/ResearchIQ>

Overall Architecture



Project Structure

```
ResearchIQ/
├── backend/                                # Python FastAPI Backend
│   ├── routers/
│   │   ├── __init__.py
│   │   └── chat.py                        # Main API endpoints
│   ├── services/
│   │   ├── __init__.py
│   │   ├── llm.py                        # AI/LLM integration (Groq)
│   │   └── papers.py                     # Academic paper fetching
│   ├── database_migrations/             # SQL migrations (if using DB)
│   ├── main.py                          # FastAPI app entry point
│   ├── requirements.txt                  # Python dependencies
│   ├── render.yaml                      # Render deployment config
│   ├── runtime.txt                      # Python version spec
│   └── .env.example                     # Environment variables template
├── frontend/                             # React TypeScript Frontend
│   ├── src/
│   │   ├── components/
│   │   │   ├── ChatInterface.tsx        # Main chat UI
│   │   │   ├── MessageBubble.tsx       # Individual messages
│   │   │   ├── DeepDiveChat.tsx        # Paper-specific chat modal
│   │   │   ├── ProjectRoadmap.tsx      # Project ideas timeline
│   │   │   ├── Dashboard.tsx          # Main dashboard
│   │   │   └── Tilt3D.tsx              # 3D card effect component
│   │   ├── context/
│   │   │   └── LibraryContext.tsx      # Global state (history, bookmarks)
│   │   ├── services/
│   │   │   └── api.ts                   # Backend API client
│   │   ├── utils/
│   │   │   └── exportPDF.ts            # PDF export functionality
│   │   ├── assets/                      # Images, icons
│   │   ├── App.tsx                      # Root component
│   │   ├── main.tsx                     # Entry point
│   │   └── index.css                    # Global styles
│   ├── package.json                     # Node dependencies
│   ├── vite.config.ts                   # Vite configuration
│   ├── tailwind.config.js               # Tailwind CSS config
│   ├── postcss.config.js               # PostCSS config
│   └── vercel.json                      # Vercel deployment config
├── .gitignore                           # Git ignore rules
└── README.md                            # Project documentation
```

Backend Architecture (FastAPI + Python)

1. Entry Point: `main.py`

Purpose: Initializes the FastAPI application and configures middleware.

Key Components:

- **FastAPI App:** Creates the main application instance
- **CORS Middleware:** Allows frontend (Vercel) to communicate with backend (Render)
- **Router Registration:** Mounts the `/api/chat` endpoint

Code Flow:

```
app = FastAPI()
app.add_middleware(CORSMiddleware, allow_origins=[])
app.include_router(chat.router, prefix="/api")
```

Allowed Origins:

- `http://localhost:5173` (local development)

- `https://research-iq-flax.vercel.app` (production)

2.API Router: `routers/chat.py`

Purpose: Handles all API endpoints and orchestrates the data pipeline.

Endpoints:

POST `/api/chat`

Request:

```
{
  "query": "machine learning healthcare"
}
```

Response:

```
{
  "original_query": "machine learning healthcare",
  "refined_query": "machine learning healthcare diagnosis",
  "papers": [...],
  "gap_analysis": {...},
  "ideas": {...}
}
```

Processing Pipeline:

1. **Query Refinement** → Calls `refine_query()` to optimize search terms
2. **Paper Fetching** → Calls `fetch_papers()` to get academic papers
3. **Paper Summarization** → Concurrently summarizes all papers using `summarize_paper()`
4. **Gap Analysis** → Calls `analyze_gaps_and_ideas()` to identify research gaps and generate project ideas

POST `/api/deep-dive`

Purpose: Allows users to ask specific questions about a single paper.

Request:

```
{
  "paper": {"title": "...", "abstract": "..."},
  "question": "What methodology was used?"
}
```

Response:

```
{
  "answer": "The paper used..."
}
```

3. LLM Service: `services/llm.py`

Purpose: Integrates with Groq's Llama 3.3 API for all AI operations.

Key Functions:

`refine_query(user_input: str) -> str`

- **Input:** Raw user query (e.g., "AI in healthcare")
- **Output:** Optimized academic search query (e.g., "artificial intelligence healthcare diagnosis")
- **Purpose:** Converts casual language into precise academic keywords

`summarize_paper(title: str, abstract: str) -> dict`

- **Input:** Paper title and abstract
- **Output:** Structured summary with 4 fields:
 - `simple_explanation`: Student-friendly 1-2 sentence summary

- `problem_statement` : Main problem addressed
- `methodology` : Approach/method used
- `key_results` : Main findings
- **Concurrency:** Uses `asyncio.gather()` to summarize multiple papers in parallel

`analyze_gaps_and_ideas(papers: list) -> dict`

- **Input:** List of 3 papers (limited to prevent token overflow)
- **Output:** Two-part analysis:
 1. **Gap Analysis:**
 - `common_approaches` : What methods are commonly used
 - `limitations` : Current limitations in the field
 - `research_gaps` : Unexplored areas
 - `suggested_improvements` : How to improve existing work
 2. **Project Ideas:**
 - `mini_projects` : 3 small project ideas with tech stacks
 - `major_project` : 1 comprehensive project idea

Fallback Mechanism: If Groq API fails (rate limit, timeout, etc.), returns hardcoded fallback ideas to prevent app crash.

4. Papers Service: `services/papers.py`

Purpose: Fetches academic papers from multiple sources with intelligent fallback.

3-Tier Fallback System:

Tier 1: Semantic Scholar (Primary)

- **URL:** `https://api.semanticscholar.org/graph/v1/paper/search`
- **Pros:** High-quality papers, rich metadata
- **Limit:** ~100 requests/minute (free tier)

Tier 2: OpenAlex (Fallback 1)

- **URL:** `https://api.openalex.org/works`
- **Pros:** Massive database (200M+ papers), no API key needed
- **Special:** Reconstructs abstracts from inverted index format

Tier 3: arXiv (Fallback 2)

- **URL:** `https://export.arxiv.org/api/query`
- **Pros:** Pre-print papers, cutting-edge research
- **Format:** XML parsing required

Data Flow:

```
papers = []
# Try Semantic Scholar first
if len(papers) < limit:
    # Fallback to OpenAlex
if len(papers) < limit:
    # Fallback to arXiv
return papers[:limit]
```

Frontend Architecture (React + TypeScript + Vite)

1. Entry Point: `main.tsx` & `App.tsx`

Purpose: Bootstrap the React application and render the root component.

Flow:

```
main.tsx → App.tsx → Dashboard.tsx → ChatInterface.tsx
```

2. Core Components

ChatInterface.tsx

Purpose: Main chat UI where users interact with the AI.

Features:

- Message history display
- Input field with send button
- Loading states with skeleton UI
- Error handling with custom messages
- "New Chat" button to reset conversation

State Management:

```
const [messages, setMessages] = useState<Message[]>([WELCOME_MESSAGE]);
const [input, setInput] = useState('');
const [isLoading, setIsLoading] = useState(false);
```

API Integration:

- Calls `sendChatQuery()` from `api.ts`
 - Handles success/error responses
 - Saves to history via `LibraryContext`
-

MessageBubble.tsx

Purpose: Renders individual messages (user or assistant) with rich content.

Features:

- **User Messages:** Simple text bubbles
- **Assistant Messages:** Complex layout with:
 - Refined query display
 - Paper cards grid (with 3D tilt effect)
 - Research gaps section
 - Project roadmap
 - PDF export button

Paper Card Features:

- Title, summary (problem/method/result)
 - Bookmark button (saves to library)
 - Deep Dive chat button
 - External link to paper
-

DeepDiveChat.tsx

Purpose: Modal for asking specific questions about a paper.

Features:

- Opens as overlay modal
 - Chat interface for paper-specific Q&A
 - Calls `/api/deep-dive` endpoint
 - 3D flip animation on open
-

ProjectRoadmap.tsx

Purpose: Visual timeline showing project progression from mini to major projects.

Features:

- Vertical timeline with connecting line
- Expandable cards (click to see details)
- Icons for each step (Zap, Code, Rocket, Trophy)
- Tech stack badges
- 3D tilt effect on hover

Data Structure:

```
{
  mini_projects: [
    {title, description, tech_stack},
    {title, description, tech_stack},
    {title, description, tech_stack}
  ],
  major_project: {
    title, description, tech_stack
  }
}
```

Tilt3D.tsx

Purpose: Reusable component that adds 3D tilt effect to any child element.

Features:

- Mouse tracking for realistic tilt
- Glare effect (light reflection)
- Configurable tilt angle, scale, speed
- Smooth transitions

Usage:

```
<Tilt3D maxTilt={10} scale={1.03} glare={true}>
  <PaperCard />
</Tilt3D>
```

3. State Management: LibraryContext.tsx

Purpose: Global state for history and bookmarks using React Context API.

Features:

- **History:** Stores all past searches with timestamps
- **Bookmarks:** Saves papers for later reference
- **Persistence:** Uses `localStorage` to persist data across sessions

API:

```
const {
  saveToHistory,      // Save a search
  history,            // Get all searches
  toggleBookmark,    // Bookmark/unbookmark a paper
  isBookmarked,      // Check if paper is bookmarked
  bookmarks           // Get all bookmarked papers
} = useLibrary();
```

4. API Service: services/api.ts

Purpose: Centralized API client for all backend calls.

Functions:

`sendChatQuery(query: string): Promise<ChatResponse>`

- **Endpoint:** POST `/api/chat`
- **Returns:** Papers, gap analysis, project ideas

`deepDiveChat(paper: Paper, question: string): Promise<{answer: string}>`

- **Endpoint:** POST `/api/deep-dive`
- **Returns:** AI answer about specific paper

Environment Configuration:

```
const API_URL = import.meta.env.VITE_API_URL || 'http://localhost:8000';
```

- **Development:** Uses `http://localhost:8000`
- **Production:** Uses `https://researchiq-backend-hbzg.onrender.com`

5. Utilities: `utils/exportPDF.ts`

Purpose: Generates PDF reports of research analysis.

Features:

- Uses `jsPDF` library
- Includes:
 - Original query
 - Refined query
 - All papers with summaries
 - Research gaps
- Auto-downloads as `ResearchIQ-{query}.pdf`

Data Flow: Complete User Journey

Scenario: User searches for "AI in healthcare"

```
1. USER INPUT
  ↳ User types "AI in healthcare" in ChatInterface
  ↳ Calls sendChatQuery("AI in healthcare")

2. FRONTEND → BACKEND
  ↳ POST /api/chat {query: "AI in healthcare"}

3. BACKEND PROCESSING
  ↳ Step 1: Refine Query
  |   ↳ LLM: "AI in healthcare" → "artificial intelligence healthcare diagnosis"
  |
  ↳ Step 2: Fetch Papers
  |   ↳ Try Semantic Scholar → Get 5 papers
  |   ↳ Try OpenAlex → Get 3 more papers
  |   ↳ Total: 8 papers
  |
  ↳ Step 3: Summarize Papers (Concurrent)
  |   ↳ Paper 1 → LLM → Summary (problem/method/result)
  |   ↳ Paper 2 → LLM → Summary
  |   ↳ ... (all 8 papers in parallel)
  |
  ↳ Step 4: Analyze Gaps & Generate Ideas
  |   ↳ Take first 3 papers
  |   ↳ LLM → Gap analysis (approaches, limitations, gaps)
  |   ↳ LLM → Project ideas (3 mini + 1 major)

4. BACKEND → FRONTEND
  ↳ Return JSON:
    {
      original_query: "AI in healthcare",
      refined_query: "artificial intelligence healthcare diagnosis",
      papers: [8 papers with summaries],
      gap_analysis: {...},
      ideas: {mini_projects: [...], major_project: {...}}
    }

5. FRONTEND RENDERING
  ↳ MessageBubble displays:
  |   ↳ "Searched: artificial intelligence healthcare diagnosis"
  |   ↳ 8 Paper Cards (with 3D tilt effect)
  |   ↳ Research Gaps section
  |   ↳ Project Roadmap (timeline)
  |
  ↳ User can:
  |   ↳ Click paper → Open Deep Dive chat
  |   ↳ Bookmark paper → Saved to Library
  |   ↳ Export PDF → Download report
  |   ↳ Expand roadmap → See project details
```

Deployment Architecture

Frontend: Vercel

- **URL:** <https://research-iq-flax.vercel.app>
- **Build:** Vite builds React app → `dist/` folder
- **Hosting:** Static files served via CDN
- **Environment:** `VITE_API_URL` points to Render backend

Backend: Render

- **URL:** <https://researchiq-backend-hbzig.onrender.com>
- **Runtime:** Python 3.11
- **Server:** Uvicorn (ASGI server)

- **Environment:** GROQ_API_KEY for AI access
- **Auto-sleep:** Free tier sleeps after 15 min inactivity

CORS Configuration

```
origins = [  
  "http://localhost:5173",           # Local dev  
  "https://research-iq-flax.vercel.app", # Production  
]
```

Key Features Implemented

Core Features

1. **Multi-source paper fetching** (Semantic Scholar, OpenAlex, arXiv)
2. **AI-powered query refinement** (Groq Llama 3.3)
3. **Concurrent paper summarization** (asyncio)
4. **Research gap analysis** (AI-generated)
5. **Project idea generation** (3 mini + 1 major)
6. **Deep dive chat** (paper-specific Q&A)
7. **PDF export** (jsPDF)
8. **History & bookmarks** (localStorage)

UI/UX Features

1. **3D tilt effect** on paper cards (Tilt3D component)
2. **Parallax background** with floating gradients
3. **Tactile send button** with 3D press effect
4. **Cinematic modal** with 3D flip animation
5. **Interactive roadmap** with expandable timeline
6. **Skeleton loading states** for better UX
7. **Custom error messages** for different failure scenarios

Technical Features

1. **TypeScript** for type safety
2. **React Context** for global state
3. **Async/await** for non-blocking I/O
4. **Error boundaries** and fallback mechanisms
5. **Environment-based configuration** (dev vs prod)
6. **CORS security** (restricted origins)
7. **Rate limit handling** (429 errors)

Security & Best Practices

API Keys

- Groq API key stored in Render environment variables
- Never exposed to frontend
- `.env` files in `.gitignore`

CORS

- Only allows specific origins (localhost + Vercel)
- Prevents unauthorized API access

Rate Limiting

- Groq free tier: ~30 RPM, ~6000 TPM
- Fallback mechanisms prevent crashes
- User-friendly error messages

Data Privacy

- No user authentication (public app)
- History stored locally (browser localStorage)
- No server-side data collection

Technology Stack Summary

Layer	Technology	Purpose
Frontend	React 18	UI framework
Frontend	TypeScript	Type safety
Frontend	Vite	Build tool
Frontend	Tailwind CSS	Styling
Frontend	Lucide React	Icons
Frontend	jsPDF	PDF generation
Backend	FastAPI	API framework
Backend	Python 3.11	Runtime
Backend	Uvicorn	ASGI server
Backend	httpx	HTTP client
AI	Groq (Llama 3.3)	LLM API
APIs	Semantic Scholar	Paper database
APIs	OpenAlex	Paper database
APIs	arXiv	Pre-print papers
Deployment	Vercel	Frontend hosting
Deployment	Render	Backend hosting

Future Enhancements (Planned)

1. **User Authentication** - Save searches across devices
2. **Database Integration** - PostgreSQL for persistent storage
3. **Advanced Search Filters** - Filter by year, citation count, etc.
4. **Collaboration Features** - Share research with team members
5. **Citation Export** - BibTeX, APA, MLA formats
6. **Mobile App** - React Native version
7. **Browser Extension** - Quick access from any webpage
8. **Advanced AI Features** - Paper recommendations, trend analysis

—